Operations

Environment, deployment, and the incident runbook. Every query here is meant to be copy-pasteable as written.

`src/lib/env.ts` is the authoritative source for configuration. Where this document disagrees with it, the schema is right.

1 Environment variables

Variable	Required	Default	Notes
`DATABASE_URL`	always	composed from `DB_*`	A shell-exported value silently beats `.env`
`TEST_DATABASE_URL`	integration tests	—	Must differ from `DATABASE_URL` and contain `test`
`SESSION_SECRET`	always	—	32+ chars. Checked twice, in `env.ts` and in middleware
`AUTH_MODE`	always	**none**	`stub` | `cgsauth`. No default, by design
`CGS_AUTH_BASE_URL`	`cgsauth` only	`https://teamcgs.io`	UAT: `https://uat.teamcgs.io`
`CGS_AUTH_API_KEY`	**when cgsauth**	—	Server-side only, never sent to the browser
`APP_URL`	—	`http://localhost:3000`	Used in email links
`NODE_ENV`	—	`development`	`development` | `test` | `production`
`EMAIL_MODE`	—	`stub`	`stub` | `ses`
`AWS_REGION`	**when ses**	—	The region the SES identity is verified in
`AWS_ACCESS_KEY_ID`	optional	—	Must pair with the secret
`AWS_SECRET_ACCESS_KEY`	optional	—	Must pair with the id
`AWS_SESSION_TOKEN`	**temporary credentials only**	—	See §2. The trap
`EMAIL_FROM`	**when ses**	—	Must exactly match a verified SES identity
`EMAIL_REPLY_TO`	—	—	e.g. an OPS Support inbox
`EMAIL_ALWAYS_CC`	—	`""`	Comma-separated. Applies to **every** message, requestor mail included

Variable	Required	Default	Notes
`DISPATCH_SECRET`	when a scheduler runs	—	16+ chars. Absence 404s the sweeper

Conditional rules the schema enforces at boot

1. **AUTH_MODE=cgsauth requires CGS_AUTH_API_KEY.** Without it every login would 401 indistinguishably from a wrong password. The app refuses to boot instead.
2. **EMAIL_MODE=ses requires AWS_REGION and EMAIL_FROM.**
3. **The AWS key pair is both-or-neither.** Exactly one is always a mistake, and a silent one — the SDK would fall through to its default chain and authenticate as somebody other than the key you supplied.
4. **AWS_SESSION_TOKEN requires the key pair.** A token alone configures nothing and means a paste went half-finished.

Each of those maps to an `addIssue` call in `env.ts`'s `superRefine`. One further behaviour is deliberately **not** enforced:

- **Leaving all three AWS variables blank under ses is valid and intentional.** Authentication then goes to the AWS SDK's default credential chain — a named profile, an SSO session, or an instance/task role. Each of those refreshes itself. **This is what a deployment should use**, which is why no check rejects it.

> `ROSTER_ALLOWLIST` and `ROSTER_ALLOW_ALL_WHEN_EMPTY` do not exist. If you find them in an older note, that note is wrong — there is no eligibility allowlist for associates. See domain §2.

2 Email through SES

The session-token trap

> **This caused a multi-hour outage. Read it before configuring SES.**

AWS access keys come in two kinds:

Prefix	Kind	Needs AWS_SESSION_TOKEN?
AKIA…	Permanent IAM user key	No — and passing one is an error
ASIA…	**Temporary** — SSO, assume-role	**Yes, always**

An `ASIA` pair without its session token authenticates as nobody. SES answers **403 on every send.**

The outage was not the 403 itself but its classification: 403 used to map to `SERVICE_UNAVAILABLE`, a *transient* code, so the dispatcher retried for hours and reported a misconfiguration as a provider outage. **403 now maps to `EMAIL_REJECTED`** — permanent — and the row is marked `failed` immediately with the provider's own exception name in `last_error`.

So today the signature of this mistake is:

```
status = 'failed'
last_error = 'EMAIL_REJECTED: UnrecognizedClientException — The security token
              included is invalid — HTTP 403'
```

Temporary credentials also expire, usually within hours. They are a local-development affordance. A deployment wants a permanent key or, better, no explicit keys at all — the unenforced case noted in §1.

Diagnosing mail that isn't arriving

Check EMAIL_MODE first. The stub queues, renders, and marks rows `sent` without delivering anything. An environment that forgot `EMAIL_MODE=ses` looks healthy in every log and every table while delivering nothing. That failure mode shows as `status='sent'` — so it never explains stuck `pending` rows.

```
-- Permanently failed. Someone was never told.
select id, recipient, template, attempts, last_error, created_at
  from notification_outbox where status = 'failed'
  order by created_at desc limit 50;

-- Stuck. Dispatch isn't running, or errors are being swallowed as transient.
select count(*), min(created_at) from notification_outbox
  where status = 'pending' and created_at < now() - interval '1 hour';
```

Status	Means
`pending`	Queued, waiting for `next_attempt_at`. Healthy in small numbers; old rows mean dispatch isn't firing
`sending`	Claimed by a run in progress. Stuck here suggests a crashed process — `attempts` was already incremented, so it still converges
`sent`	Delivered, **or captured by the stub**. `sent_at` and `provider_message_id` both set
`failed`	Permanent and terminal. `last_error` holds `code: detail`

A permanent failure also logs, at error level:

```
[email:dispatch] permanent failure id=… reason=…
```

That line is **the only push signal in the system.** No admin UI surfaces it.

The dev probe

`GET /api/dev/email/probe?to=<address>` sends one real message through the app's actual configured transport and reports back `emailMode`, `region`, `from`, the credential **key prefix** (`ASIA` vs `AKIA`, never the key itself), and whether a session token is present.

It 404s in production. It is for diagnosing a staging environment configured the same way, not a live outage.

Retrying a failed row after fixing the cause

```
update notification_outbox
   set status = 'pending', attempts = 0, next_attempt_at = now(), last_error =
null
 where id = '…';
```

3 Migrations and seeds

Order is load-bearing.

```
pnpm db:migrate   # first, always
pnpm db:seed      # admin whitelist, then fleet roster
```

pnpm db:migrate must run before pnpm db:seed. The admin-whitelist seed reads `roster_events`, a table created by the newest migration (`20260831175400_roster_management`). If it's missing, the seed fails loudly with `42P01 relation "roster_events" does not exist` — not silently.

pnpm db:seed must run before anyone can manage the whitelist. An unseeded database has no `admin_whitelist` rows, so `super_admin` is false for everyone and `/roster`'s Admins tab is invisible — including to whoever the seed would have made a holder. Fail-closed, and it reads exactly like a bug to whoever deploys.

Re-running the seeds reverts `/roster` edits

Both seeds re-assert their own fields on every run. This is deliberate drift correction, and it has a consequence worth knowing before someone edits a seeded row in the UI:

Seed	Re-asserts	Never touches
Admin whitelist	`full_name, email, domain_id, site, super_admin`	`active`, for any row a human managed via `/roster`
Fleet roster	Van `plate`, `car_type`, `site`; driver `mobile`, `site`, `shift`	`active`, ever

So a `/roster` edit to a **seeded** admin's name, email, Domain ID, site, or `super_admin` flag **is reverted** on the next `pnpm db:seed`. Only activate/deactivate survives. An edit that should stick belongs in `src/modules/auth/admin-whitelist-seed.ts`, not just in the UI.

The `active` carve-out exists because without it every deploy would silently un-retire everyone an admin had retired.

Migrating a database that has real data

One migration can abort a deploy, and it cannot be pre-cleared the obvious way. `20260827162542` adds `assigned_van_id` and the five `rental_*` columns **and** `reservations_approved_van_check` — which requires every approved row to carry a van or a rental — in a single `up()`, with no backfill between them.

Every pre-existing row therefore has `assigned_van_id` and `rental_plate` NULL when the constraint is validated, so **every** `status = 'approved'` row violates it and the whole migration aborts. (The later `20260830060413_add_reassigned_status` migration only ever widens its checks for exactly this reason, and its docblock names this one as the contrast.)

Pre-flight — this runs *before* the migration, because it touches no new column:

```
select count(*) from reservations where status = 'approved';
```

A non-zero count is the number of rows that will abort the run. Do not reach for the columns the migration adds; they do not exist yet, so a query naming `assigned_van_id` fails with `column ... does not exist` rather than returning a count.

Three ways forward, in order of preference:

1. **Run the migration against a staging copy of production data first.** The abort is loud, atomic and non-corrupting, so this tells you the blast radius safely.
2. **Split it** into add-columns → backfill → add-constraint. That creates a real window in which rows can be assigned before the constraint lands.
3. **Move the affected rows off** `approved` before migrating and restore them after assigning vans. Runnable today, but it rewrites real status history.

None of this applies to a fresh database, where there are no approved rows.

Separately, `20260827225113_remap_retired_purposes` is a no-op on a fresh database and **required** against real data, where all five retired purpose values are in use. See domain §6.

4 The outbox sweeper

Mail is enqueued during a request and delivered separately. Two things drain the queue, and they run the same function:

after() — every write route calls `dispatchOutbox` post-response, so the requestor never waits on SES. Because the claim query takes the oldest due rows regardless of who queued them, a write opportunistically drains anything else pending too.

The sweeper — `GET /api/internal/dispatch-outbox`. This is **the only thing that fires when nobody is writing**, which is exactly what you need for mail queued while SES was down when no new bookings are arriving.

Configuring it

```
DISPATCH_SECRET=<16+ chars>
```

Then schedule a call every **5–15 minutes**:

```
curl -H "x-dispatch-secret: $DISPATCH_SECRET" \
     https://<host>/api/internal/dispatch-outbox
```

The endpoint is idempotent, so a more frequent schedule is harmless.

DISPATCH_SECRET	Wrong/missing header	Behaviour
Unset	—	**404.** Unconfigured means unavailable, not unguarded
Set	Missing or wrong	401
Set	Correct	Drains up to 25 rows, returns `{sent, failed, retrying}`

Retry behaviour

- **Claim** is `FOR UPDATE SKIP LOCKED`, which is load-bearing: it stops a concurrent sweeper and `after()` — or two app instances — from double-sending.
- **attempts increments at claim time**, not after the send, so a row orphaned in `sending` by a killed process still converges toward the cap.
- **Backoff** is `min(2^attempts, 60)` minutes — doubling, capped at an hour.
- **MAX_ATTEMPTS = 8.** A transient failure past the cap is marked `failed`.
- **Permanent codes** (`EMAIL_REJECTED`, `VALIDATION_FAILED`) skip retry entirely. So do render errors — an unknown template or invalid payload is a programming error, not a transient one.
- **An unknown thrown error is treated as transient** and retried, because discarding a notification over a bug in error handling is the worse failure.

5 Monitoring

Nothing alerts today. The sweeper returns a JSON summary and calls nothing; the dispatcher writes `console.error` on a permanent failure and nothing else; there is no monitoring integration in the tree. The queries in §2 are run by hand.

Worth adding, in priority order:

1. **pending rows older than an hour.** The strongest signal that delivery has stopped.
2. **Any failed row.** Permanent means somebody was never told.
3. **EMAIL_MODE not being ses in production.** This produces *zero* error signal by itself — every log and every table looks healthy — which is what makes it the highest-value alert to add.
4. The `[email:dispatch] permanent failure` log line, if logs ship somewhere with alerting.

6 Production readiness

Auth

- [ ] `AUTH_MODE=cgsauth` with the real `CGS_AUTH_API_KEY` in an untracked environment. The stub accepts any password unconditionally.
- [ ] `pnpm db:seed` run against **every** environment, so the whitelist is populated and `/roster` is usable.
- [ ] Decide the role-revocation strategy. Roles ride in the session JWT for up to **8 hours** — deactivating a whitelist row does not take effect until next login or expiry. No session table exists. See §7.
- [ ] `dev-identities.ts` and stub mode deleted once cgsauth is the only path.

Email

- [ ] `EMAIL_MODE=ses`, with `AWS_REGION` and `EMAIL_FROM` matching a verified identity.
- [ ] Credentials are a **permanent** key or the default credential chain — not an `ASIA` pair that expires (§2).
- [ ] SES sender identity verified with DKIM, and the account out of the SES sandbox.
- [ ] `DISPATCH_SECRET` set **and** a scheduler actually calling the sweeper. Without it, only `after()` delivers, so anything queued during an outage never retries.
- [ ] `EMAIL_ALWAYS_CC` is the intended list — it copies every message, including the requestor's own.

Database

- [ ] Managed Postgres provisioned, backups configured.
- [ ] `pnpm db:migrate` before `pnpm db:seed` (§3).
- [ ] Pre-flight query for `20260827162542` run against real data (§3).

Platform

- [ ] `pnpm typecheck && pnpm lint && pnpm test && pnpm test:int && pnpm build` green.
- [ ] Elevance Sans licensed and installed. **Neither Elevance Sans nor Inter is loaded today** — there is no `next/font` call and no `@font-face`, so the app renders in the OS system font.
- [ ] Designer sign-off on the password-reveal icon fill. Figma specifies `#C7D2D6` (1.54:1), below the 3:1 WCAG 1.4.11 minimum; it ships as `--color-gray-2` (5.74:1) instead, and the deviation needs approving.

7 Known hazards

Roles lag revocation by up to 8 hours. `SESSION_TTL_SECONDS` is 28800 and the role is carried in the JWT. Immediate revocation needs a session table or a short TTL with refresh. The `super_admin` flag is exempt — it is re-read from the database on every `/api/admins` request — but the base `admin_support` role is not.

middleware.ts must not become proxy.ts by rename. `middleware` is deprecated in Next 16, but it runs on the **edge runtime**, and that constraint is what forces the auth gate to avoid importing Kysely or `pg`. `proxy.ts` runs on Node, so the same code there would let a `pg` import leak into the route gate and fail at request time instead of at build time. **The migration must land together with an import-restriction lint rule for that file.**

A van can be double-booked. Once drivers and vans became independently assignable, nothing checks for overlapping assignments. A product gap, not a deploy blocker.

`/roster` van numbering is typed by hand. No auto-numbering for `VAN-00N`.

See also

- Local setup — local development

- Domain rules — the business rules
- Architecture — how the code is laid out
- Email and notifications — the notification pipeline and the templates